SPRINT 3 RETROSPECTIVE DOCUMENT

Project: TripRoute (Travel Assistant)

Team 5 Members:

- Sameer Maayiz
- Dongwan Kim
- Mainuddin

Overview

Sprint 3 marked a significant transformation of the TripRoute project, delivering a complete end-to-end trip planning system. The sprint successfully migrated from ChatGPT to Gemini AI, achieving dramatic performance improvements (10-30 seconds reduced to 1-2 seconds) while gaining access to real-time Google Search grounding for accurate place information. The team built a production-ready FastAPI server with session management, developed a comprehensive frontend interface with interactive map visualization, and enhanced route optimization to support all four transportation modes. The system now provides a seamless user experience from initial chat interaction through AI-powered recommendations to optimized multi-modal route display.

Successes & achievements

What went well?

1. Successfully migrated from OpenAI ChatGPT to Google Gemini 2.5 Flash Lite with 95% reduction in response time (from 10-30s to 1-2s).

2. Implemented Google Search grounding providing real-time, accurate place data including ratings, reviews, hours, and Place IDs for reliable route optimization.

3. Built production-ready FastAPI server with in-memory session management, enabling concurrent multi-user support.

4. Developed complete frontend interface (demo.html) with responsive 3-column layout combining chat, map, and route details in a single cohesive view.

5. Enhanced route optimization engine to support all 4 transportation modes with special handling for transit (initial driving optimization, then leg-by-leg transit directions).

6. Implemented Place ID normalization system for reliable matching between Gemini recommendations and Google Maps data.

7. Created comprehensive documentation suite (SETUP_GUIDE.md, RUNNING_LOCALLY.md, TROUBLESHOOTING.md, etc.) for smooth onboarding and deployment.

8. Achieved seamless integration between Gemini structured extraction and Google Maps APIs through grounding metadata Place ID extraction.

9. Implemented streaming responses for both chat and structured extraction, providing real-time feedback to users.

10. Successfully validated all 4 transportation modes with real-world test cases across multiple cities.

Challenges & blockers

What didn't go well?

- Initial Gemini grounding prompts were too permissive, allowing AI to skip Google Maps queries and use training data, resulting in missing Place IDs.
- Place ID extraction from grounding metadata required complex parsing of Google Maps URLs (cid parameter vs place_id parameter).
- Transit mode optimization required special two-step approach (driving first for waypoint order, then transit leg-by-leg) due to Google Maps API limitations.
- Frontend map rendering occasionally delayed when Gemini grounding returned large numbers of places simultaneously.
- Session storage in-memory means all data is lost on server restart (acceptable for Phase 1, but requires DynamoDB migration for production).
- CORS configuration initially blocked frontend-backend communication during local testing with file:// protocol.
- Google Maps API rate limits occasionally throttled requests during heavy testing with multiple concurrent sessions.
- Frontend CSS required significant iteration to achieve proper responsive layout with draggable splitters across different screen sizes.

Lessons learned & next steps

What can be improved?

1. Implement persistent session storage using AWS DynamoDB for Phase 2 production deployment.

2. Add caching layer for Google Maps Place Details API to reduce redundant queries and API costs.

3. Implement API key usage monitoring and rate limiting to prevent quota exhaustion.

4. Add automated testing suite (pytest for backend, Jest for frontend) to catch regressions early.

5. Create CI/CD pipeline for automated deployment to AWS EC2 with health checks.

6. Implement user authentication system for personalized trip history and saved itineraries.

7. Add error boundaries and retry logic for Gemini API failures and rate limit handling.

8. Optimize frontend map rendering by implementing progressive place marker loading (render in batches instead of all at once).

9. Add analytics tracking to understand user behavior and optimize UX (time spent in chat vs map, most common transport modes, etc.).

10. Create staging environment separate from production for safe integration testing.

11. Implement structured logging with timestamps and session IDs for better debugging and monitoring.

12. Add input validation and sanitization for user messages to prevent injection attacks.

Team member contributions & reflections

SAMEER MAAYIZ

- Developed complete frontend interface (demo.html) with responsive 3-column layout integrating chat, map, and route visualization.
- Implemented draggable column splitters for customizable user experience.
- Created chat UI with streaming message support and loading indicators.
- Integrated Google Maps JavaScript API with dynamic marker placement and route polylines.
- Built transport mode switcher with seamless route updates.
- Designed and implemented print-friendly itinerary view.
- Learned advanced CSS Grid and Flexbox techniques for complex responsive layouts.
- Suggested implementing WebSocket connections for future real-time collaborative trip planning.
- Reflected that early design mockups in Figma significantly accelerated frontend development and reduced revision cycles.

DONGWAN KIM

- Built production FastAPI server (server.py) with session management and background tasks.
- Migrated from OpenAI ChatGPT API to Google Gemini with streaming support.
- Implemented Gemini Google Search grounding integration for real-time place data.
- Created Pydantic models for structured data extraction (TripRouteRecommendations, TripMeta, RecommendationItem).
- Developed /chat and /optimize API endpoints with proper error handling.
- Configured CORS middleware for cross-origin frontend-backend communication.
- Implemented background task for asynchronous structured extraction while user continues chatting.
- Created comprehensive environment configuration with api.env management.
- Proposed implementing AWS Lambda functions for serverless deployment in Phase 3.
- Reflected that thorough API documentation and request/response examples reduced frontend-backend integration time.

MAINUDDIN

- Enhanced route optimization engine (optimization.py) to support all 4 transportation modes.
- Implemented Place ID normalization for reliable matching between Gemini and Google Maps data.
- Developed special transit handling (two-step optimization: driving first, then transit leg-by-leg).
- Integrated Google Maps Place Details API for place enrichment (address, hours, ratings, reviews).
- Created structured direction output with leg-by-leg navigation steps.
- Developed step normalization for handling nested transit steps (bus, subway, train details).
- Wrote comprehensive documentation suite (6+ markdown files) covering setup, installation, troubleshooting, and features.
- Tested route optimization across multiple cities and transportation modes.
- Proposed implementing isochrone mapping for "places reachable within X minutes" feature.
- Reflected that modular code architecture with clear separation (chat, extraction, optimization) made parallel development efficient and reduced merge conflicts.